

PRACTICAL - 1

AIM: Installation of Code Standards and Libraries used in Reinforcement Learning (Python/Keras/Tensorflow).

INTRODUCTION:

Reinforcement Learning (RL) is a branch of Machine Learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties. Python is widely used for RL due to its rich ecosystem of libraries.

Key libraries include:

- **NumPy** for numerical computations
- **Matplotlib** for visualization
- **TensorFlow / Keras** for deep learning-based RL
- **Gymnasium (OpenAI Gym)** for RL environments

SOFTWARE REQUIREMENTS:

- Operating System: Windows / Linux / macOS
- Python Version: Python 3.8 or above
- IDE: VS Code / PyCharm / Jupyter Notebook

LIBRARIES USED:

- numpy
- matplotlib
- tensorflow
- keras
- gymnasium

INSTALLATION COMMANDS:

```
pip install numpy
pip install matplotlib
pip install tensorflow
pip install keras
pip install gymnasium
```

```
PS C:\Users\HP\Desktop\Daiz > pip install matplotlib
Requirement already satisfied: matplotlib in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (3.10.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (1.26.4)
Requirement already satisfied: packaging>=20.0 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (23.2)
Requirement already satisfied: pillow>=8 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

PS C:\Users\HP\Desktop\Daiz > pip install tensorflow
Collecting tensorflow
  Downloading tensorflow-2.20.0-cp310-cp310-win_amd64.whl.metadata (4.6 kB)
Requirement already satisfied: absl-py>=1.0.0 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from tensorflow) (2.3.1)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Requirement already satisfied: flatbuffers>=24.3.25 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from tensorflow) (25.12.19)
Collecting gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
```

OUTPUT:

```
RL.py
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import tensorflow as tf
4 import gymnasium as gym
5
6 print("NumPy version:", np.__version__)
7 print("TensorFlow version:", tf.__version__)
8 print("Gymnasium installed successfully")
9
```

```
2026-02-08 21:30:31.169990: I tensorflow/core/util/port.cc:153] oneDNN custom operations
s from different computation orders. To turn them off, set the environment variable `T
NumPy version: 1.26.4
TensorFlow version: 2.20.0
Gymnasium installed successfully
```

PRACTICAL - 2

AIM: Write a program to solve Multi Armed Bandit Problem.

INTRODUCTION:

The Multi-Armed Bandit problem is a classic Reinforcement Learning problem where an agent must choose between multiple actions (slot machine arms), each with an unknown reward distribution.

The ϵ -greedy approach balances:

- **Exploration:** trying new arms
- **Exploitation:** choosing the best-known arm

ALGORITHM:

1. Initialize estimated values for all arms to zero
2. For each iteration:
 - With probability ϵ , select a random arm
 - Otherwise, select the arm with the highest estimated reward
3. Receive reward and update the estimate
4. Repeat for fixed iterations

PROGRAM:

```
import numpy as np
import random

true_rewards = [0.2, 0.5, 0.75]

# Actual (unknown) success probabilities of each arm
num_arms = len(true_rewards)

# Total number of arms
estimated_values = np.zeros(num_arms)

# Estimated reward of each arm (initially 0)
arm_counts = np.zeros(num_arms)

# Number of times each arm is selected
epsilon = 0.1

# Exploration probability (10% chance of exploration, 90% exploitation)
total_reward = 0
```



```
# Total reward collected

iterations = 1000

# Total number of times the agent will act
for t in range(iterations):

    # Decide whether to explore or exploit
    if random.random() < epsilon:

        arm = random.randint(0, num_arms - 1)

        # Exploration: choose a random arm
    else:

        arm = np.argmax(estimated_values)

        # Exploitation: choose the best estimated arm
    reward = 1 if random.random() < true_rewards[arm] else 0

    # Generate reward (1 = success, 0 = failure)

    # STEP 5: Update values
    arm_counts[arm] += 1

    # Increase count of selected arm
    old_value = estimated_values[arm]

    # Store old estimated value
    estimated_values[arm] = old_value + (reward - old_value) /
arm_counts[arm]

    # Update estimated value using incremental mean formula
    total_reward += reward

    # Add reward to total

# -----

print("Estimated values of arms:", estimated_values)
print("Number of times each arm was selected:", arm_counts)
print("Total reward collected:", total_reward)
print("Best arm according to agent:", np.argmax(estimated_values))
```

OUTPUT:

```
PS C:\Users\saifa> & "C:/Program Files/Python312/python.exe" "c:/Users/saifa/6th Sem/Lab/RL/Cod
e/exp.py"
Estimated values of arms: [0.28947368 0.58139535 0.76278564]
Number of times each arm was selected: [ 38.  43. 919.]
Total reward collected: 737
Best arm according to agent: 2
PS C:\Users\saifa>
```

PRACTICAL - 3

AIM: Write a program to implement Dynamic programming algorithms for solving MDPs (Markov Decision Process), Policy Evaluation, Policy Iteration, Policy Improvement and Value Iteration.

INTRODUCTION:

In many real-world situations, an agent (robot, software system, student model, game player, etc.) has to **take decisions step-by-step in an uncertain environment**.

Each decision changes the state of the system and gives a **reward or penalty**.

This type of sequential decision-making problem is formally modeled as a **Markov Decision Process (MDP)**.

The problem is:

To find an **optimal policy** (best sequence of actions) that maximizes the **expected cumulative reward over time**.

What is a Markov Decision Process (MDP)?

An MDP is defined by **five components**:

- **S** – Set of states
- **A** – Set of actions
- **P** – Transition probability $P(s' | s, a)$
- **R** – Reward function $R(s, a, s')$
- **γ (gamma)** – Discount factor ($0 \leq \gamma \leq 1$)

Meaning:

- The agent is in some state **s**
- It takes an action **a**
- It moves to a new state **s'** with some probability
- It receives a **reward**
- Future rewards are discounted by **γ**

The **Markov property** means:

The next state depends only on the current state and action, not on past history.

POLICY EVALUATION:

It answers:

“If the agent follows this policy, how good is each state?”

It computes the **value function $V(s)$** for a fixed policy using the Bellman expectation equation

#CODE:

```
# =====
# SIMPLE MDP USING DYNAMIC PROGRAMMING
# =====

# States
```



```
states = ['A', 'B']

# Discount factor
gamma = 0.9

# Initial policy (randomly chosen)
policy = {'A': 'left'}

# Initial state values
V = {'A': 0, 'B': 0}

print("Initial Policy:", policy)
print("Initial Values:", V)

# -----
# POLICY ITERATION
# -----
print("\n--- POLICY ITERATION ---")

for iteration in range(3):

    # ---- Policy Evaluation ----
    for i in range(5): # repeat to stabilize values
        if policy['A'] == 'right':
            V['A'] = 10 + gamma * V['B']
        else:
            V['A'] = 0 + gamma * V['A']

    # ---- Policy Improvement ----
    left_value = 0 + gamma * V['A']
    right_value = 10 + gamma * V['B']

    if right_value > left_value:
        policy['A'] = 'right'
    else:
        policy['A'] = 'left'

    print("Iteration {iteration+1}")
    print(" Policy:", policy)
```



```
print(" Values:", V)

# -----
# VALUE ITERATION
# -----
print("\n--- VALUE ITERATION ---")

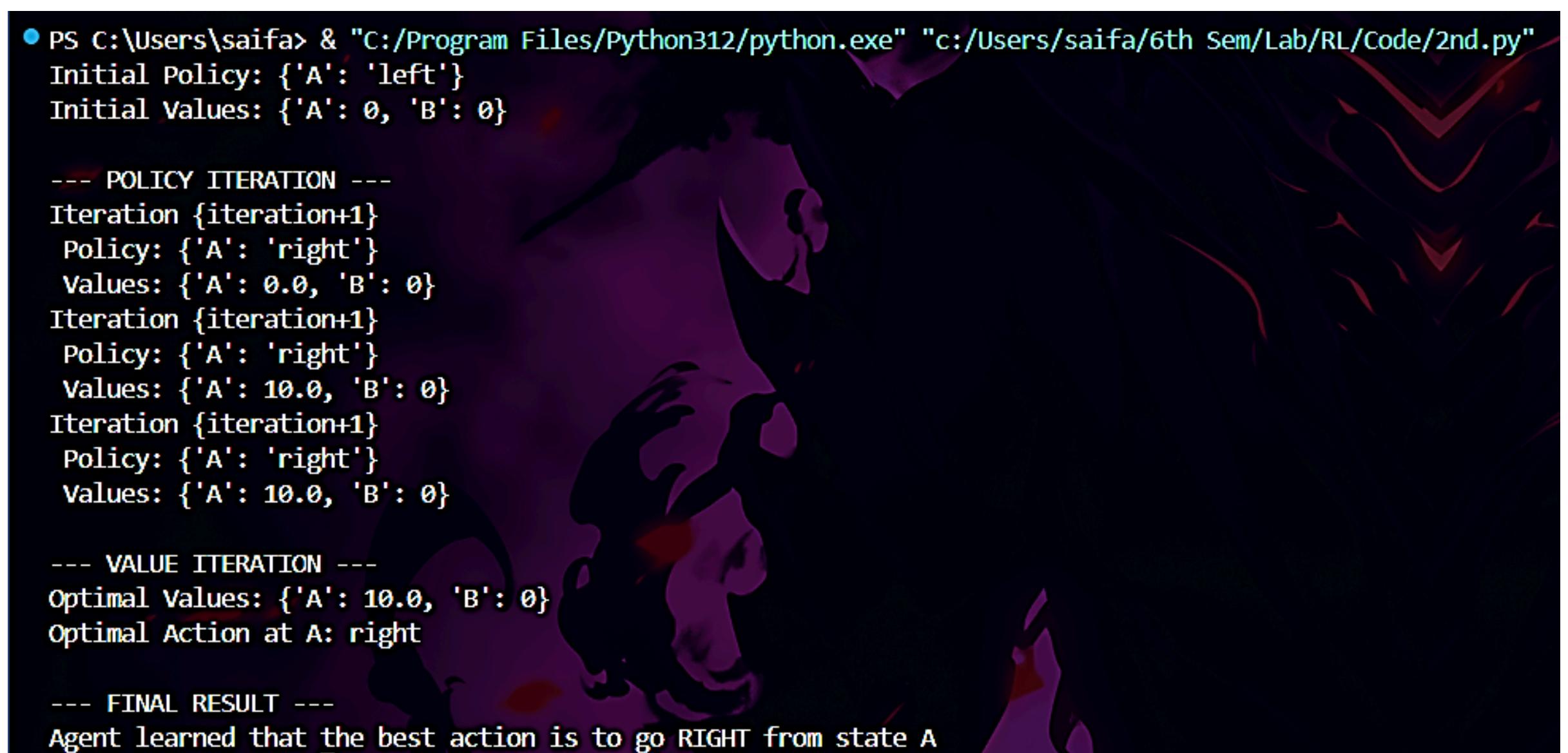
V = {'A': 0, 'B': 0}

for i in range(5):
    V['A'] = max(
        0 + gamma * V['A'],    # left
        10 + gamma * V['B']    # right
    )

print("Optimal Values:", V)
print("Optimal Action at A: right")

# -----
# FINAL RESULT
# -----
print("\n--- FINAL RESULT ---")
print("Agent learned that the best action is to go RIGHT from state A")
```

OUTPUT :



```
PS C:\Users\saifa> & "C:/Program Files/Python312/python.exe" "c:/Users/saifa/6th Sem/Lab/RL/Code/2nd.py"
Initial Policy: {'A': 'left'}
Initial Values: {'A': 0, 'B': 0}

--- POLICY ITERATION ---
Iteration {iteration+1}
Policy: {'A': 'right'}
Values: {'A': 0.0, 'B': 0}
Iteration {iteration+1}
Policy: {'A': 'right'}
Values: {'A': 10.0, 'B': 0}
Iteration {iteration+1}
Policy: {'A': 'right'}
Values: {'A': 10.0, 'B': 0}

--- VALUE ITERATION ---
Optimal Values: {'A': 10.0, 'B': 0}
Optimal Action at A: right

--- FINAL RESULT ---
Agent learned that the best action is to go RIGHT from state A
```


PRACTICAL - 4

AIM: Write a program to implement Dynamic programming algorithms for Policy Evaluation, Policy Iteration and Value Iteration.

INTRODUCTION:

In many real-world situations, an agent (robot, software system, student model, game player, etc.) has to **take decisions step-by-step in an uncertain environment**.

Each decision changes the state of the system and gives a **reward or penalty**.

This type of sequential decision-making problem is formally modeled as a **Markov Decision Process (MDP)**.

The problem is:

To find an **optimal policy** (best sequence of actions) that maximizes the **expected cumulative reward over time**.

What is a Markov Decision Process (MDP)?

An MDP is defined by **five components**:

- **S** – Set of states
- **A** – Set of actions
- **P** – Transition probability $P(s' | s, a)$
- **R** – Reward function $R(s, a, s')$
- **γ (gamma)** – Discount factor ($0 \leq \gamma \leq 1$)

Meaning:

- The agent is in some state **s**
- It takes an action **a**
- It moves to a new state **s'** with some probability
- It receives a **reward**
- Future rewards are discounted by **γ**

The **Markov property** means:

The next state depends only on the current state and action, not on past history.

POLICY EVALUATION:

It answers:

“If the agent follows this policy, how good is each state?”

It computes the **value function $V(s)$** for a fixed policy using the Bellman expectation equation

#CODE:

```
# =====
# SIMPLE 5-STATE MDP USING DP
# =====

# States
states = ['A', 'B', 'C', 'D', 'E']
```

```
# Discount factor
gamma = 0.9

# Initial random policy (only left/right)
policy = {
    'A': 'left',
    'B': 'left',
    'C': 'left',
    'D': 'left'
}

# Initial state values
V = {s: 0 for s in states}

print("Initial Policy:", policy)
print("Initial Values:", V)

# -----
# POLICY ITERATION
# -----
print("\n--- POLICY ITERATION ---")

for iteration in range(3):

    # ---- Policy Evaluation ----
    for _ in range(5): # stabilize values
        for i in range(len(states) - 1):
            s = states[i]
            next_s = states[i + 1]

            if policy[s] == 'right':
                V[s] = 10 + gamma * V[next_s]
            else:
                V[s] = 0 + gamma * V[s]

    # ---- Policy Improvement ----
    for i in range(len(states) - 1):
        s = states[i]
```



```
    next_s = states[i + 1]

    left_value = 0 + gamma * V[s]
    right_value = 10 + gamma * V[next_s]

    if right_value > left_value:
        policy[s] = 'right'
    else:
        policy[s] = 'left'

    print(f"\nIteration {iteration + 1}")
    print(" Policy:", policy)
    print(" Values:", V)

# -----
# VALUE ITERATION
# -----
print("\n--- VALUE ITERATION ---")

V = {s: 0 for s in states}

for _ in range(5):
    for i in range(len(states) - 1):
        s = states[i]
        next_s = states[i + 1]

        V[s] = max(
            0 + gamma * V[s],    # left
            10 + gamma * V[next_s] # right
        )

    print("Optimal Values:", V)
    print("Optimal Action: RIGHT from all states except E")

# -----
# FINAL RESULT
# -----
print("\n--- FINAL RESULT ---")
print("Agent learned to move RIGHT to reach terminal state E")
```

OUTPUT :

```
PS C:\Users\saifa> & "C:/Program Files/Python312/python.exe" "c:/Users/saifa/6th Sem/Lab/RL/Code/3rd.py"
Initial Values: {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}

--- POLICY ITERATION ---

Iteration 1
Policy: {'A': 'right', 'B': 'right', 'C': 'right', 'D': 'right'}
Values: {'A': 0.0, 'B': 0.0, 'C': 0.0, 'D': 0.0, 'E': 0}

Iteration 2
Policy: {'A': 'right', 'B': 'right', 'C': 'right', 'D': 'right'}
Values: {'A': 34.39, 'B': 27.1, 'C': 19.0, 'D': 10.0, 'E': 0}

Iteration 3
Policy: {'A': 'right', 'B': 'right', 'C': 'right', 'D': 'right'}
Values: {'A': 34.39, 'B': 27.1, 'C': 19.0, 'D': 10.0, 'E': 0}

--- VALUE ITERATION ---
Optimal Values: {'A': 34.39, 'B': 27.1, 'C': 19.0, 'D': 10.0, 'E': 0}
Optimal Action: RIGHT from all states except E

--- FINAL RESULT ---
Agent learned to move RIGHT to reach terminal state E
```

PRACTICAL - 5

AIM: Write a program to implement Grid-world by using Policy Iteration.

INTRODUCTION:

In reinforcement learning, an agent interacts with an environment by taking actions step-by-step. Each action changes the state of the agent and provides a reward or penalty. Such sequential decision-making problems are commonly represented using a Grid World environment. The objective is to find an optimal policy that enables the agent to reach the terminal state with minimum cumulative cost.

GRID WORLD AND MDP:

Grid World can be modeled as a Markov Decision Process (MDP) consisting of:

- S – Set of states (grid cells)
- A – Set of actions (Up, Down, Left, Right)
- P – State transition function
- R – Reward function
- γ (gamma) – Discount factor

Meaning: The agent is in a state, performs an action, moves to the next state, receives a reward, and future rewards are discounted by γ . The Markov property states that the next state depends only on the current state and action.

POLICY ITERATION:

Policy Iteration is a dynamic programming algorithm used to determine the optimal policy. Policy Evaluation calculates the value of each state for a fixed policy using the Bellman Expectation Equation:

$$V(s) = R(s,a) + \gamma V(s')$$

Policy Improvement updates the policy by selecting the action that maximizes the expected value using the Bellman Optimality Equation. The evaluation and improvement steps are repeated until the policy becomes optimal and stable.

CODE :

```
import numpy as np

# =====
# Grid World Parameters
# =====

GRID_SIZE = 5
ACTIONS = ['U', 'D', 'L', 'R']
gamma = 0.9      #<--- important for convergence
```



```
theta = 0.01      # convergence threshold

# =====
# Initialization
# =====
V = np.zeros((GRID_SIZE, GRID_SIZE))
policy = np.random.choice(ACTIONS, (GRID_SIZE, GRID_SIZE))

terminal_states = [(0, 0), (4, 4)]

# =====
# Transition Function
# =====
def step(state, action):
    i, j = state

    if action == 'U':
        i = max(i - 1, 0)
    elif action == 'D':
        i = min(i + 1, GRID_SIZE - 1)
    elif action == 'L':
        j = max(j - 1, 0)
    elif action == 'R':
        j = min(j + 1, GRID_SIZE - 1)

    return (i, j)

# =====
# Policy Iteration
# =====
is_policy_stable = False

while not is_policy_stable:

    # ----- POLICY EVALUATION -----
    for _ in range(100):      # safety limit
        delta = 0

        for i in range(GRID_SIZE):
            for j in range(GRID_SIZE):

                if (i, j) in terminal_states:
```

```

        continue

    old_value = V[i, j]
    action = policy[i, j]
    next_state = step((i, j), action)

    reward = -1
    V[i, j] = reward + gamma * V[next_state]

    delta = max(delta, abs(old_value - V[i, j]))

    if delta < theta:
        break

# ----- POLICY IMPROVEMENT -----
is_policy_stable = True

for i in range(GRID_SIZE):
    for j in range(GRID_SIZE):

        if (i, j) in terminal_states:
            continue

        old_action = policy[i, j]
        action_values = {}

        for action in ACTIONS:
            next_state = step((i, j), action)
            action_values[action] = -1 + gamma * V[next_state]

        best_action = max(action_values, key=action_values.get)
        policy[i, j] = best_action

        if old_action != best_action:
            is_policy_stable = False

# =====
# Output
# =====
print("Optimal Value Function:\n")
print(np.round(V, 2))

```

```
print("\nOptimal Policy:\n")
print(policy)
```

OUTPUT :

```
● PS C:\Users\saifa\6th Sem\Lab\RL\Code> & "C:/Program Files/Python312/python.exe"
Optimal Value Function:

[[ 0.   -1.   -1.9 -2.71 -3.44]
 [-1.   -1.9 -2.71 -3.44 -2.71]
 [-1.9 -2.71 -3.44 -2.71 -1.9 ]
 [-2.71 -3.44 -2.71 -1.9  -1.  ]
 [-3.44 -2.71 -1.9  -1.   0.  ]]

Optimal Policy:

[['U' 'L' 'L' 'L' 'D']
 ['U' 'U' 'U' 'U' 'D']
 ['U' 'U' 'U' 'D' 'D']
 ['U' 'U' 'D' 'D' 'D']
 ['U' 'R' 'R' 'R' 'D']]
```


PRACTICAL - 6

AIM: Write a program to perform Monte Carlo Prediction, Monte Carlo Off-Policy Control Importance Sampling.

INTRODUCTION:

Monte Carlo methods in reinforcement learning learn value functions using complete episodes. The agent interacts with the environment and updates values based on actual returns received at the end of each episode. These methods do not require knowledge of transition probabilities.

Monte Carlo Prediction evaluates a given policy, while Off-Policy Control uses importance sampling to learn the optimal policy from a different behavior policy.

MONTÉ CARLO METHODS:

- Episode-based learning
- No bootstrapping
- Uses average returns

Meaning:

The agent generates episodes and updates values using total rewards. Importance sampling helps adjust differences between behavior and target policies.

FORMULA:

$V(s) = \text{average of returns}$

$Q(s,a) = Q(s,a) + W(G - Q(s,a))$

Where:

- G – Return
- W – Importance weight

CODE :

```
import gym, random
from collections import defaultdict

env = gym.make("FrozenLake-v1", is_slippery=False)
gamma = 0.9
```

```
# MC Prediction
def mc_pred():
    V = defaultdict(float)
    returns = defaultdict(list)

    for _ in range(500):
        s, _ = env.reset()
        ep = []

        while True:
            a = random.randint(0,3)
            ns, r, done, _, _ = env.step(a)
            ep.append((s, r))
            s = ns
            if done: break

        G = 0
        for s, r in reversed(ep):
            G = gamma * G + r
            returns[s].append(G)
            V[s] = sum(returns[s]) / len(returns[s])

    return V
```

```
# Off-policy with importance sampling
def mc_off():
    Q = defaultdict(lambda: [0]*4)
    C = defaultdict(lambda: [0]*4)

    for _ in range(500):
        s, _ = env.reset()
        ep = []

        while True:
            a = random.randint(0,3)
            ns, r, done, _, _ = env.step(a)
            ep.append((s, a, r))
            s = ns
            if done: break

        G, W = 0, 1

        for s, a, r in reversed(ep):
```

```
G = gamma * G + r
C[s][a] += W
Q[s][a] += (W/C[s][a]) * (G - Q[s][a])
```

```
if a != Q[s].index(max(Q[s])): break
W = W * 4 # 1/0.25
```

```
return Q
```

```
V = mc_pred()
```

```
Q = mc_off()
```

```
print("\n--- STATE VALUES (V) ---")
```

```
for s in sorted(V):
```

```
    print(f"State {s:2} : {V[s]:.4f}")
```

```
print("\n--- ACTION VALUES (Q) ---")
```

```
for s in sorted(Q):
```

```
    print(f"State {s:2} : {[':.3f}'.format(x) for x in Q[s]]}")
```

OUTPUT :


```
--- STATE VALUES (V) ---
State 0 : 0.0103
State 1 : 0.0076
State 2 : 0.0171
State 3 : 0.0000
State 4 : 0.0122
State 6 : 0.0431
State 8 : 0.0207
State 9 : 0.0623
State 10 : 0.1568
State 13 : 0.1682
State 14 : 0.5420

--- ACTION VALUES (Q) ---
State 1 : ['0.000', '0.000', '0.000', '0.000']
State 2 : ['0.000', '0.000', '0.000', '0.000']
State 3 : ['0.000', '0.000', '0.000', '0.000']
State 4 : ['0.590', '0.656', '0.000', '0.000']
State 6 : ['0.000', '0.810', '0.000', '0.000']
State 8 : ['0.000', '0.000', '0.729', '0.590']
State 9 : ['0.000', '0.810', '0.810', '0.000']
State 10 : ['0.000', '0.900', '0.000', '0.686']
State 13 : ['0.000', '0.000', '0.900', '0.000']
State 14 : ['0.000', '0.000', '1.000', '0.810']
PS C:\Users\saifa\6th Sem\Lab\RL\Code>
```


PRACTICAL - 7

AIM: Write a program to implement SARSA and Q-Learning in a grid-world or Frozen Lake environment based on Temporal Difference Learning.

INTRODUCTION:

In reinforcement learning, Temporal Difference (TD) learning updates value estimates using current rewards and next state estimates without waiting for the episode to end. It combines ideas of Monte Carlo and Dynamic Programming.

SARSA and Q-Learning are TD control algorithms used to learn optimal policies. The agent interacts with the environment, updates Q-values step-by-step, and improves its decisions over time to maximize rewards.

SARSA AND Q-LEARNING:

Both methods use Q-values:

- SARSA – On-policy learning
- Q-Learning – Off-policy learning

Meaning:

SARSA updates using the action actually taken, while Q-Learning updates using the best possible action.

UPDATE RULES:

SARSA:

$$Q(s,a) = Q(s,a) + \alpha [R + \gamma Q(s', a') - Q(s,a)]$$

Q-Learning:

$$Q(s,a) = Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s,a)]$$

Where:

- α – Learning rate
- γ – Discount factor
- R – Reward

CODE :

```
import numpy as np
import random
```

```
# Grid size
n_states = 4
```

```
n_actions = 4 # 0=up,1=down,2=left,3=right

# Q tables
Q_sarsa = np.zeros((n_states, n_actions))
Q_q = np.zeros((n_states, n_actions))

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 1000

# simple environment
def step(state, action):
    if action == 0: # up
        next_state = max(0, state-1)
    elif action == 1: # down
        next_state = min(n_states-1, state+1)
    elif action == 2: # left
        next_state = max(0, state-1)
    else: # right
        next_state = min(n_states-1, state+1)

    reward = 1 if next_state == n_states-1 else 0
    done = next_state == n_states-1
    return next_state, reward, done

def choose_action(Q, state):
    if random.uniform(0,1)<epsilon:
        return random.randint(0,3)
    return np.argmax(Q[state])

# ----- SARSA -----
for ep in range(episodes):

    state = 0
    action = choose_action(Q_sarsa, state)

    done = False
    while not done:

        next_state, reward, done = step(state, action)
        next_action = choose_action(Q_sarsa, next_state)
```

```
        Q_sarsa[state][action] += alpha * (
            reward + gamma * Q_sarsa[next_state][next_action] -
            Q_sarsa[state][action]
        )

        state = next_state
        action = next_action

# ----- Q Learning -----
for ep in range(epochs):

    state = 0
    done = False

    while not done:

        action = choose_action(Q_q, state)
        next_state, reward, done = step(state, action)

        Q_q[state][action] += alpha * (
            reward + gamma * np.max(Q_q[next_state]) -
            Q_q[state][action]
        )

        state = next_state

print("SARSA Q-table")
print(Q_sarsa)

print("\nQ-Learning Q-table")
print(Q_q)
```

OUTPUT :


```
PS C:\Users\saiifa\6th Sem\Lab\RL\Code> & "C:/Program Files/Python37/Python.exe" RL.py
SARSA Q-table
[[0.67792267 0.77625633 0.68099689 0.77027633]
 [0.65579407 0.8557871 0.67347794 0.87723219]
 [0.7499098 1. 0.74946471 0.99753497]
 [0. 0. 0. 0. ]]

Q-Learning Q-table
[[0.72842843 0.80683563 0.72727855 0.81 ]
 [0.72692188 0.9 0.72758786 0.89733172]
 [0.80082812 1. 0.80825621 0.99800332]
 [0. 0. 0. 0. ]]

PS C:\Users\saiifa\6th Sem\Lab\RL\Code>
```

PRACTICAL - 8

AIM: Write a program to implement Cliff Walking problem using Q-learning.

INTRODUCTION:

In reinforcement learning, an agent learns to make decisions by interacting with an environment over a sequence of steps. At each step, the agent observes its current state, selects an action, receives a reward or penalty, and moves to the next state. The goal of the agent is to learn an optimal policy that maximizes cumulative reward over time. The Cliff Walking problem is a classical example used to demonstrate Temporal Difference learning and action-value methods such as Q-learning.

CLIFF WALKING AND MDP:

Cliff Walking is modeled as a Markov Decision Process (MDP) consisting of:

- **S** – Set of states (4×12 grid cells)
- **A** – Set of actions (Up, Down, Left, Right)
- **P** – State transition function
- **R** – Reward function
- **γ (gamma)** – Discount factor

Meaning: The agent starts from a fixed start state and aims to reach the goal state. Some grid cells represent a cliff region. If the agent steps into the cliff, it receives a large negative reward and returns to the start state. Each normal step gives a small negative reward to encourage the shortest path. The Markov property states that the next state depends only on the current state and chosen action.

Q-LEARNING (TEMPORAL DIFFERENCE LEARNING):

Q-learning is a model-free Temporal Difference learning algorithm used to learn the optimal action-value function. It updates the Q-value using the Bellman Optimality Equation:

$$Q(s,a) = Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s,a)]$$

Where:

- **α (alpha)** – Learning rate
- **γ (gamma)** – Discount factor
- **$\max Q(s', a')$** – Maximum estimated future reward

CODE :

```
import numpy as np
import random

# Environment Setup
ROWS = 4
COLS = 12
ACTIONS = 4 # 0=Up, 1=Down, 2=Left, 3=Right

START = (3, 0)
GOAL = (3, 11)

# Hyperparameters
alpha = 0.1 # Learning rate
gamma = 0.9 # Discount factor
epsilon = 0.1 # Exploration rate
episodes = 1000 # Increased training episodes

# Q-table initialization
Q = np.zeros((ROWS, COLS, ACTIONS))

# To track learning progress
episode_rewards = []

# Move Function
def move(state, action):
    r, c = state

    # Take action
    if action == 0: # Up
        r = max(r - 1, 0)
    elif action == 1: # Down
        r = min(r + 1, ROWS - 1)
    elif action == 2: # Left
        c = max(c - 1, 0)
    elif action == 3: # Right
        c = min(c + 1, COLS - 1)

    # Check Cliff
    if r == 3 and 1 <= c <= 10:
        return START, -100

    # Check Goal
    if (r, c) == GOAL:
```



```
        return (r, c), 100 # Better positive reward

    return (r, c), -1 # Step penalty

# Training Loop
for ep in range(epochs):
    state = START
    total_reward = 0

    while state != GOAL:
        r, c = state

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[r, c])

        next_state, reward = move(state, action)
        nr, nc = next_state

        # Q-learning update rule
        best_next_action = np.max(Q[nr, nc])
        Q[r, c, action] += alpha * (
            reward + gamma * best_next_action - Q[r, c, action]
        )

        state = next_state
        total_reward += reward

    episode_rewards.append(total_reward)

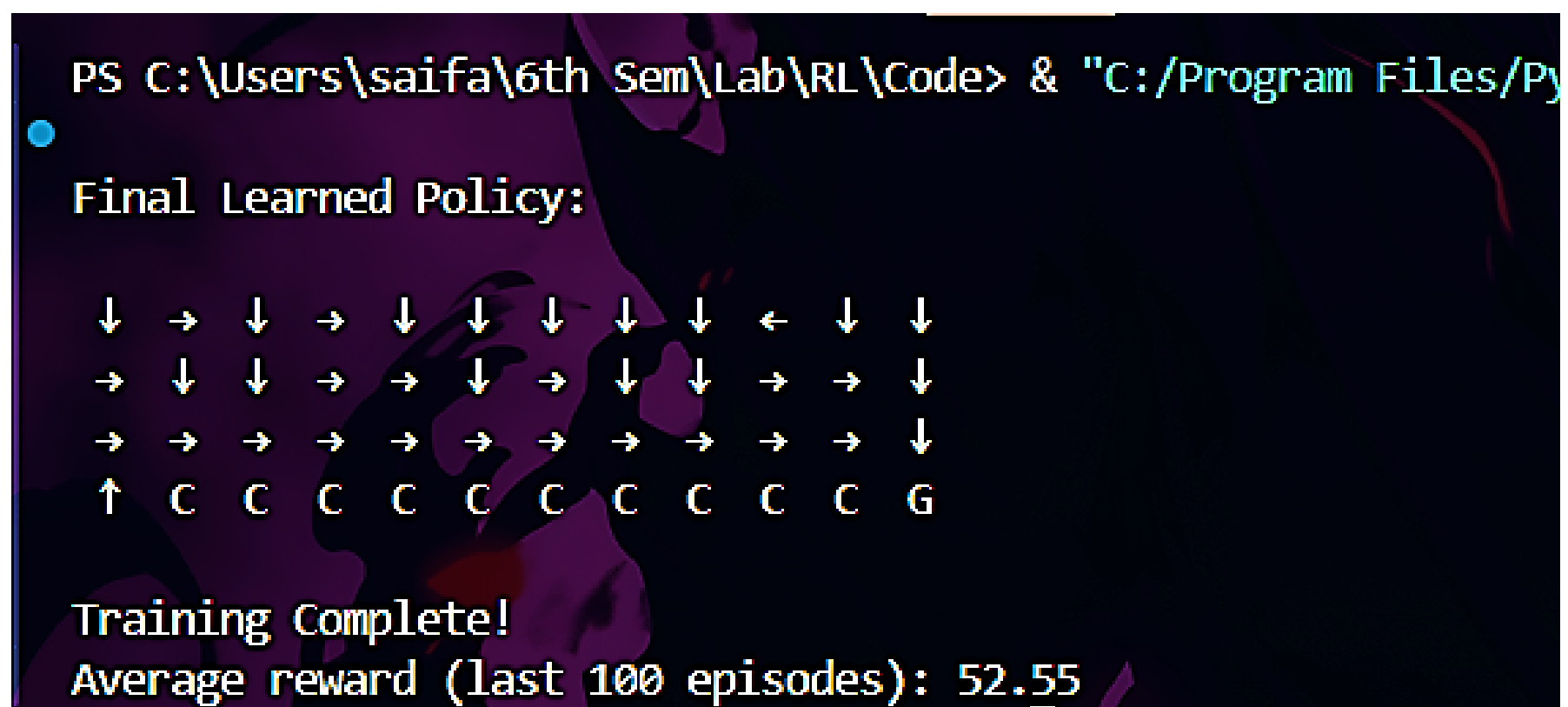
# Display Final Policy
policy = np.argmax(Q, axis=2)

action_symbols = {
    0: "↑",
    1: "↓",
    2: "←",
    3: "→"
}

print("\nFinal Learned Policy:\n")
```

```
for r in range(ROWS):
    for c in range(COLS):
        if (r, c) == GOAL:
            print(" G ", end="")
        elif r == 3 and 1 <= c <= 10:
            print(" C ", end="") # Cliff
        else:
            print(f" {action_symbols[policy[r, c]]} ", end="")
    print()

# Extra Information
print("\nTraining Complete!")
print("Average reward (last 100 episodes):",
      np.mean(episode_rewards[-100:]))
```

OUTPUT :

```
PS C:\Users\saifa\6th Sem\Lab\RL\Code> & "C:/Program Files/Py
Final Learned Policy:

↓ → ↓ → ↓ ↓ ↓ ↓ ← ↓ ↓
→ ↓ ↓ → → ↓ → ↓ ↓ → → ↓
→ → → → → → → → → → ↓
↑ C C C C C C C C C C G

Training Complete!
Average reward (last 100 episodes): 52.55
```


PRACTICAL - 10

AIM: Write a program to implement a simple DQN agent using PyTorch or TensorFlow.

INTRODUCTION: In deep reinforcement learning, an agent uses a neural network to learn optimal actions instead of a Q-table. It observes the state, takes an action, receives a reward, and moves to the next state. The goal is to maximize cumulative reward over time.

DQN SYSTEM:

A Deep Q-Network (DQN) includes:

- Neural Network – Predicts Q-values
- Replay Memory – Stores experiences
- Target Network – Stabilizes learning

Meaning:

The agent uses the network to choose actions and learns from past experiences by training on random samples.

DQN ALGORITHM:

$$Q(s,a)=Q(s,a)+\alpha[R+\gamma\max_{a'}Q_{\text{target}}(s',a')-Q(s,a)]$$

Where:

- α – Learning rate
- γ – Discount factor
- R – Reward

CODE :

```
import gym
import torch
import random
import torch.nn as nn
import torch.optim as optim
from collections import deque

env = gym.make("CartPole-v1")

model = nn.Sequential(
    nn.Linear(4, 24),
    nn.ReLU(),
    nn.Linear(24, 2)
)
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.MSELoss()

memory = deque(maxlen=2000)
epsilon = 1.0
scores = []

for episode in range(50):
    state = env.reset()[0]
    total_reward = 0

    for t in range(200):

        # choose action
        if random.random() < epsilon:
            action = env.action_space.sample()
        else:
            action = torch.argmax(model(torch.FloatTensor(state))).item()

        # take action
        next_state, reward, done, _, _ = env.step(action)

        # store experience
        memory.append((state, action, reward, next_state, done))

        state = next_state
        total_reward += reward

    if done:
        break

    scores.append(total_reward)

# training
if len(memory) > 32:
    batch = random.sample(memory, 32)

    for s, a, r, ns, d in batch:
        s = torch.FloatTensor(s)
        ns = torch.FloatTensor(ns)

        target = r if d else r + 0.99 * torch.max(model(ns)).item()

        output = model(s)
```

```
output[a] = target
```

```
loss = loss_fn(model(s), output.detach())
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

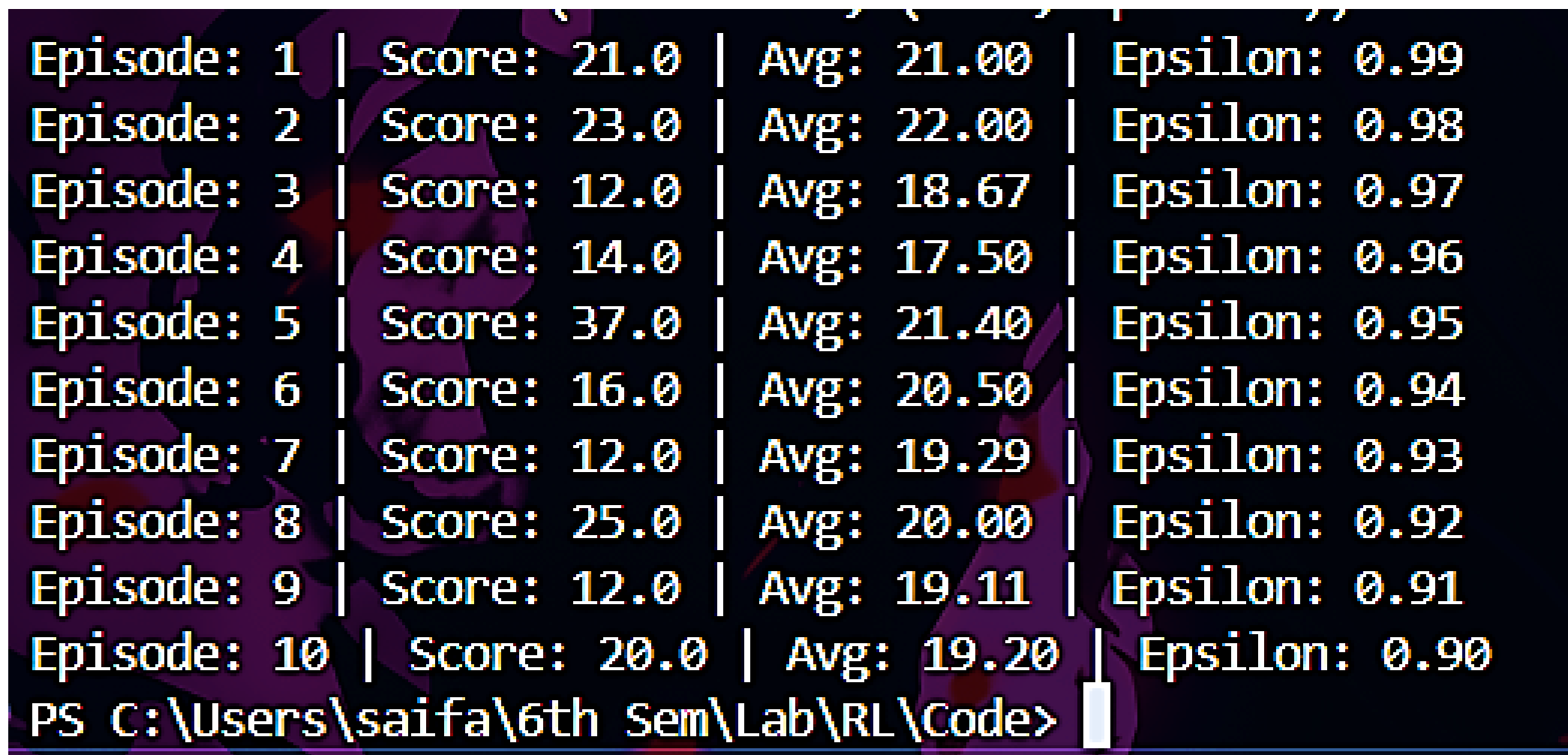
```
epsilon *= 0.99
```

```
avg_score = sum(scores[-10:]) / len(scores[-10:])
```

```
print(f"Episode: {episode+1} | Score: {total_reward} | Avg:  
{avg_score:.2f} | Epsilon: {epsilon:.2f}")
```

```
env.close()
```

OUTPUT :



A terminal window with a dark background and yellow text. It displays the results of 10 training episodes. Each line shows the episode number, the current score, the average score over the last 10 episodes, and the current epsilon value. The epsilon value decreases from 0.99 to 0.90 over the 10 episodes. The terminal prompt is 'PS C:\Users\saiifa\6th Sem\Lab\RL\Code>'.

Episode	Score	Avg	Epsilon
1	21.0	21.00	0.99
2	23.0	22.00	0.98
3	12.0	18.67	0.97
4	14.0	17.50	0.96
5	37.0	21.40	0.95
6	16.0	20.50	0.94
7	12.0	19.29	0.93
8	25.0	20.00	0.92
9	12.0	19.11	0.91
10	20.0	19.20	0.90

PS C:\Users\saiifa\6th Sem\Lab\RL\Code>

PRACTICAL - 11

AIM: Write a program to implement Smart Traffic Light Controller using Reinforcement Learning.

INTRODUCTION: In reinforcement learning, an agent learns by interacting with its environment. At each step, it observes the current state, takes an action, receives a reward or penalty, and moves to the next state. The goal is to learn an optimal policy that maximizes cumulative reward.

A Smart Traffic Light Controller uses reinforcement learning to control signals dynamically based on traffic density instead of fixed timers. The system reduces vehicle waiting time and traffic congestion.

SMART TRAFFIC LIGHT SYSTEM AND MDP:

The problem is modeled as a Markov Decision Process (MDP):

- **S** – States (Low, Medium, High traffic)
- **A** – Actions (Green for North-South or East-West)
- **P** – State transition probability
- **R** – Reward function (Less waiting = positive reward)
- **γ** – Discount factor

Meaning:

The agent observes traffic density and selects which direction gets the green signal. If congestion reduces, it gets a positive reward; otherwise, it gets a penalty. The next state depends only on the current state and chosen action (Markov property).

Q-LEARNING:

Q-learning is a model-free Temporal Difference learning algorithm that updates Q-values using:

$$Q(s,a)=Q(s,a)+\alpha[R+\gamma\max_{a'}Q(s',a')-Q(s,a)]$$

Where:

- **α** – Learning rate
- **γ** – Discount factor
- **R** – Immediate reward

CODE :

```
import numpy as np
import random
```

```
# Parameters
alpha = 0.1    # Learning rate
gamma = 0.9    # Discount factor
epsilon = 0.1  # Exploration rate
episodes = 5000

# States:
# 0 = Low NS, Low EW
# 1 = High NS, Low EW
# 2 = Low NS, High EW
# 3 = High NS, High EW

n_states = 4
n_actions = 2 # 0 = Green NS, 1 = Green EW

# Initialize Q-table
Q = np.zeros((n_states, n_actions))

# Reward Function
def get_reward(state, action):
    if state == 1 and action == 0:
        return 10
    elif state == 2 and action == 1:
        return 10
    elif state == 3:
        return 5
    else:
        return -5

# Random next state generator
def get_next_state():
    return random.randint(0, 3)

# Training using Q-learning
for episode in range(episodes):
    state = random.randint(0, 3)

    for step in range(10):

        # Epsilon-greedy action selection
        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, 1)
        else:
            action = np.argmax(Q[state])
```

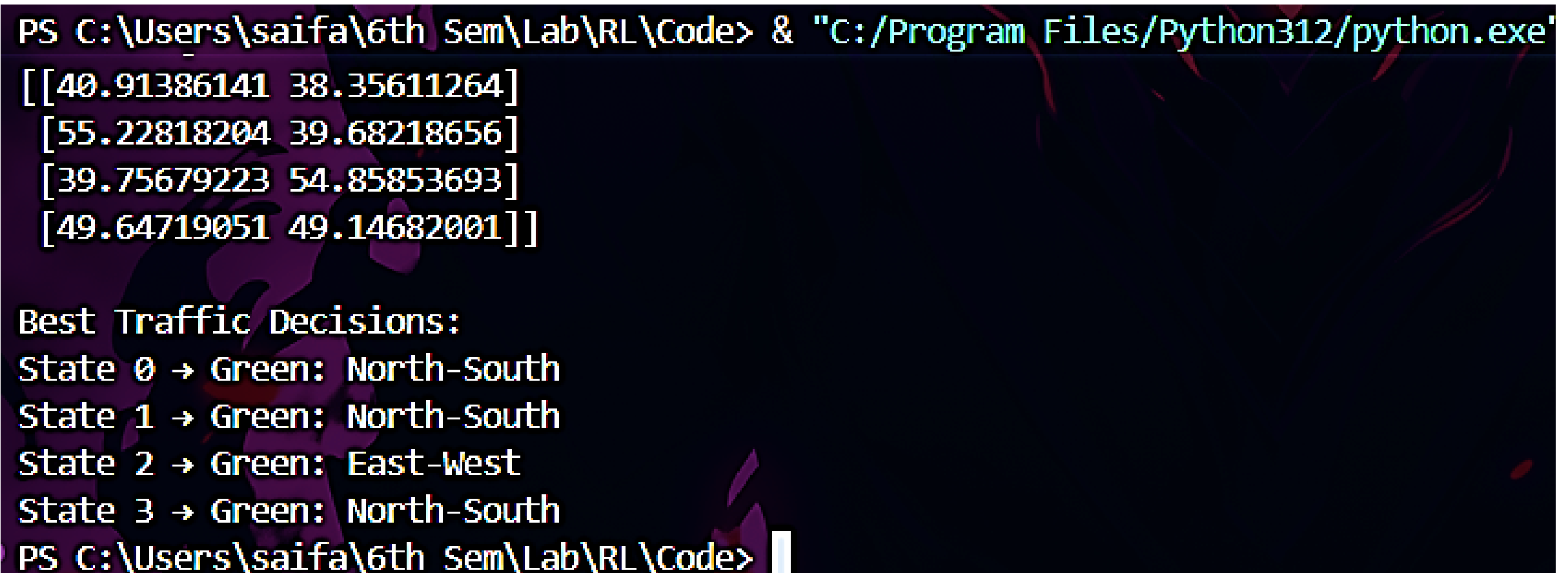
```
reward = get_reward(state, action)
next_state = get_next_state()

# Q-learning update
Q[state, action] = Q[state, action] + alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state,
action]
)

state = next_state # move to next state

# Results
print("Trained Q-Table:")
print(Q)

print("\nBest Traffic Decisions:")
for state in range(4):
    action = np.argmax(Q[state])
    if action == 0:
        print(f"State {state} → Green: North-South")
    else:
        print(f"State {state} → Green: East-West")
```

OUTPUT:

```
PS C:\Users\saiifa\6th Sem\Lab\RL\Code> & "C:/Program Files/Python312/python.exe"
[[40.91386141 38.35611264]
 [55.22818204 39.68218656]
 [39.75679223 54.85853693]
 [49.64719051 49.14682001]]

Best Traffic Decisions:
State 0 → Green: North-South
State 1 → Green: North-South
State 2 → Green: East-West
State 3 → Green: North-South
PS C:\Users\saiifa\6th Sem\Lab\RL\Code>
```